

Carrera: Ingeniería de Software con IA

MÓDULOS Y PAQUETES PARA MACHINE LEARNING CON PYTHON



RESULTADO DE APRENDIZAJE:

Explica los fundamentos de Machine Learning, diferenciando sus principales paradigmas, componentes, etapas de desarrollo y aplicaciones, y relaciona el uso de SciPy, Scikit-Learn y PyTorch con diferentes necesidades dentro del ecosistema de Inteligencia Artificial.

CONTENIDO:

IA, ML y DL; programación tradicional vs. ML; dataset, features, target, modelos, parámetros e hiperparámetros; aprendizaje supervisado, no supervisado y por refuerzo; clasificación, regresión y clustering; entrenamiento, validación y prueba; generalización, overfitting y underfitting; ciclo de vida de ML; aplicaciones; ecosistema Python; SciPy, Scikit-Learn y PyTorch.

| Saberes previos

¿Qué es un dato?

Unidad mínima de información sin procesar.
Ejemplo: 25, 30, femenino, soltero, 15.4

¿Qué es un variable?

Propiedad o atributo medible que cambia de valor.
Ejemplo: Edad, tamaño, peso, direccion, pais)

¿Qué es un dataset?

Tabla o conjunto organizado de datos estructurado en filas y columnas.

¿Qué es una variable numérica y una categórica?

Numérica: Mide cantidades para hacer cálculos (ejemplo: precio, estatura)

Categórica: Clasifica en grupos o cualidades (ejemplo: color, estado civil).

| Saberes previos

¿Por qué limpiamos los datos?

Para corregir errores, duplicados y valores faltantes, asegurando análisis precisos.

¿Por qué escalamos los datos?

Para igualar el rango de diferentes variables y evitar que valores grandes dominen a los pequeños.

¿Qué información podemos obtener de un Dataset?

Patrones, tendencias, relaciones entre variables y predicciones futuras.

Del análisis de datos a la predicción



Todo proyecto de Machine Learning recorre esta misma ruta: de la materia prima (datos) hasta una predicción útil para tomar decisiones.

*Si tenemos miles de registros históricos,
¿podemos lograr que una computadora
encuentre patrones y realice predicciones sin
escribir manualmente todas las reglas?*

Inteligencia Artificial

La **Inteligencia Artificial (IA)** es un campo de la informática que desarrolla sistemas capaces de realizar tareas que normalmente requieren inteligencia humana, como aprender, razonar, resolver problemas, comprender el lenguaje y reconocer patrones.

- **Objetivo:** que un sistema perciba su entorno, razone sobre él y actúe o decida en consecuencia.
- **Percepción:** interpretar datos del entorno (texto, imagen, sonido, sensores).
- **Razonamiento:** procesar esa información para inferir relaciones.
- **Decisión:** elegir una acción o salida a partir del razonamiento.
- **Aprendizaje:** mejorar el desempeño a partir de la experiencia aquí es donde entra el Machine Learning.

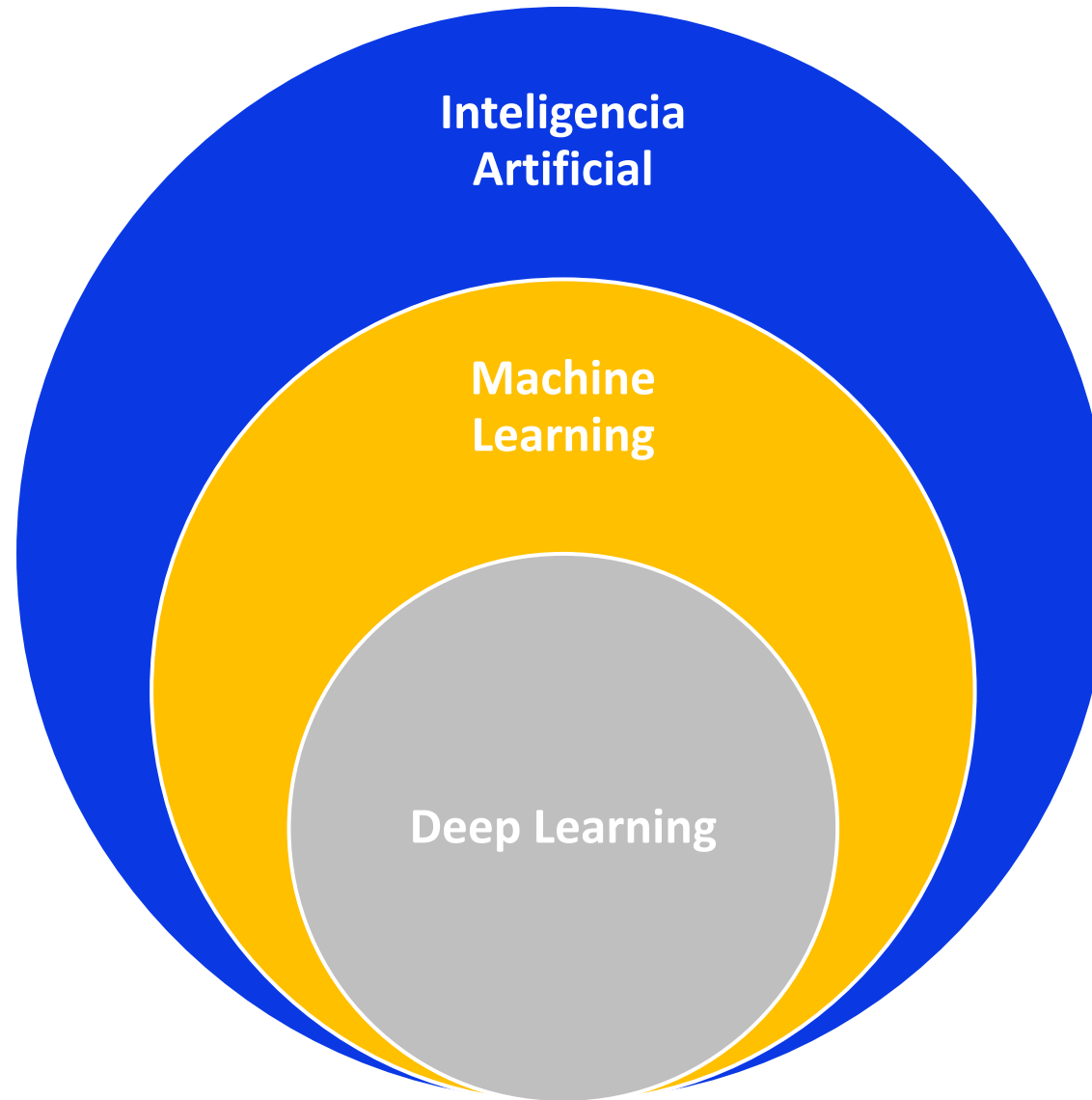


Machine Learning y Deep Learning

Sistemas que perciben,
razonan y deciden

Aprende patrones a partir de
datos, sin reglas explícitas

Redes neuronales de muchas
capas para representaciones
complejas



*Todo **Deep Learning** es **Machine Learning**,
y todo **Machine Learning** es **Inteligencia Artificial**.*

Machine Learning y Deep Learning



IA Generativa



Texto

Redacción, resúmenes, chatbots



Imágenes

Ilustraciones y diseño generado



Audio

Voz y música sintética



Código

Generación y asistencia de programación

Dos formas distintas de llegar a un resultado



Programación tradicional es:

Si edad ≥ 18

ENTONCES "Mayor de edad".

1. Desarrolla un algoritmo
2. Implementa un algoritmo en código
3. Entrega un software o solución

Machine Learning: el modelo aprende de la regla



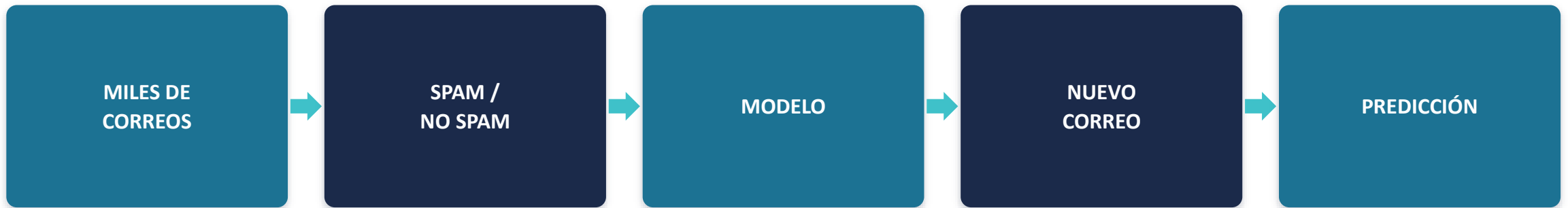
1. Recolecta y prepara los datos
2. Experimenta con diferentes algoritmos para obtener el mejor modelo
3. Entrega una predicción

***Después:** NUEVOS DATOS + MODELO → PREDICCIÓN. El modelo generaliza lo aprendido a observaciones que nunca vio.*

Tradicional vs. ML

Programación tradicional	Machine Learning
Reglas explícitas escritas a mano	Patrones aprendidos automáticamente
El programador define las reglas	El modelo aprende las reglas a partir de los datos
Datos = resultado	Datos + resultado = modelo
Útil cuando las reglas son relativamente conocidas	Útil cuando las relaciones son difíciles de especificar manualmente

Ejemplo: detección de spam



Enfoque tradicional: reglas fijas como “gratis” = spam, “premio” = spam.

Enfoque ML: el modelo descubre por sí mismo qué combinaciones de palabras son señal de spam.

El ciclo de ajuste: así “aprende” un modelo



Función, error y optimización

IDEA MATEMÁTICA

Ecuación 1:

$$\hat{y} = f(X)$$

Ecuación 2:

$$\omega_{nuevo} = \omega_{actual} - \alpha \frac{\partial L}{\partial \omega}$$

Ecuación 3:

$$Error = L(y, \hat{y})$$

Aprender, en este contexto, significa reducir sistemáticamente el error a partir de datos ya conocidos.

- **Función:** la relación matemática que el modelo usa para transformar entradas (X) en una predicción ($\hat{y}$).
- **Parámetros:** los valores internos que el modelo ajusta durante el entrenamiento.
- **Función de pérdida (L):** mide qué tan lejos está la predicción del valor real o sea mide qué tan mal se comporta el modelo.
- **Optimización:** el proceso que ajusta los parámetros para reducir el error.
- **Error:** Es la diferencia cuantitativa exacta (el margen de desviación) entre el valor proyectado por la IA y el valor real que se esperaba obtener.

¿Qué es FEATURE?

Es la información que usas para adivinar.

¿Qué es Target?

Es lo que quieres adivinar o descubrir.

¿Qué es Algoritmo?

Un **algoritmo** es una secuencia ordenada, clara y finita de pasos o instrucciones que se siguen para resolver un problema, realizar un cálculo o completar una tarea.

Features y target

Ejemplo:

Cliente - Features	Cliente - Target
Edad	-
Ingreso	-
Hijos	-
Educación	-
-	Aprobado (sí / no)

Parámetros vs. hiperparámetros

Parámetro	Hiperparámetro
Lo aprende el modelo durante el entrenamiento	Lo configura el desarrollador antes de entrenar
Se modifica automáticamente en cada iteración	Se define o se busca antes / durante el ajuste
Ejemplo: pesos de una red neuronal	Ejemplo: max_depth de un árbol de decisión
Ejemplo: coeficientes de una regresión	Ejemplo: n_estimators de un ensamble

Paradigmas de aprendizaje



Supervisado

Aprende con datos etiquetados: X (features) + y (target conocido)



No supervisado

Aprende solo con X , buscando patrones o estructura sin etiquetas



Por refuerzo

Un agente aprende por prueba y error, guiado por recompensas

Paradigmas de aprendizaje



Aprendizaje supervisado

Se entrena al modelo con **datos etiquetados**, es decir, ejemplos que ya tienen la respuesta correcta. El objetivo es que el algoritmo aprenda la relación entre los datos de entrada y la salida para poder predecirla en datos nuevos.

Clasificación: $X \rightarrow \text{categoría}$	Regresión: $X \rightarrow \text{número}$
Spam / No spam	Precio de una vivienda
Fraude / No fraude	Salario esperado
Aprobado / Rechazado	Demanda de un producto
Positivo / Negativo (sentimiento)	Temperatura estimada

Aprendizaje no supervisado

Se entrena al modelo con **datos sin etiquetar**. El algoritmo debe encontrar por sí mismo **patrones, estructuras o agrupaciones** en los datos. Es como si te dieran un montón de piezas de un puzzle y tuvieras que encontrar por tu cuenta cómo encajan.

Clustering	Reducción de dimensionalidad
Agrupar observaciones similares sin etiquetas previas	Reduce el número de variables conservando la información relevante
Algoritmo típico: K-Means	Técnica típica: PCA (Análisis de Componentes Principales)
Aplicación: segmentación de clientes	Aplicación: visualización de datos de alta dimensión

Aprendizaje por refuerzo

El modelo (llamado “agente”) **aprende interactuando con un entorno** a través de prueba y error. Recibe **recompensas o penalizaciones** según sus acciones, y su objetivo es maximizar la recompensa total. Es como enseñar a un perro: si hace algo bien, le das una golosina; si lo hace mal, no.



El agente prueba acciones sobre un entorno y ajusta su comportamiento según la recompensa recibida, sin un dataset etiquetado de antemano.

Particiones y propósitos distintos



¿Porqué no debemos entrenar el modelo con los mismos datos que usaremos para evaluarlo?

*se produce un problema llamado **overfitting***

Particiones y propósitos distintos

Desempeño en TRAIN	Desempeño en TEST
Overfitting = excelente	Overfitting = malo
Underfitting = malo	Underfitting = malo
Modelo ideal = bueno	Modelo ideal = bueno

Particiones y propósitos distintos

Underfitting - Subajuste

El **underfitting** ocurre cuando el modelo es **demasiado simple** y no logra aprender adecuadamente los patrones importantes de los datos.

Comportamiento

- Bajo rendimiento en entrenamiento.
- Bajo rendimiento en prueba.
- El modelo no captura suficientemente la relación entre las variables.

Ejemplo

Intentar predecir el precio de una vivienda utilizando únicamente: **Precio = 100 000 + 500 × Área**

Cuando existen muchos otros factores importantes como ubicación, habitaciones, antigüedad, etc

Particiones y propósitos distintos

Overfitting - Sobreajuste

El **overfitting** ocurre cuando el modelo aprende **demasiado específicamente los datos de entrenamiento**, llegando incluso a aprender ruido o particularidades que no se generalizan a nuevos datos.

Comportamiento

- El modelo parece excelente durante el entrenamiento, pero falla cuando recibe datos nuevos.

Ejemplo:

Un estudiante memoriza las respuestas exactas de los ejercicios que practicó, pero cuando cambia ligeramente el problema ya no sabe resolverlo.

Particiones y propósitos distintos

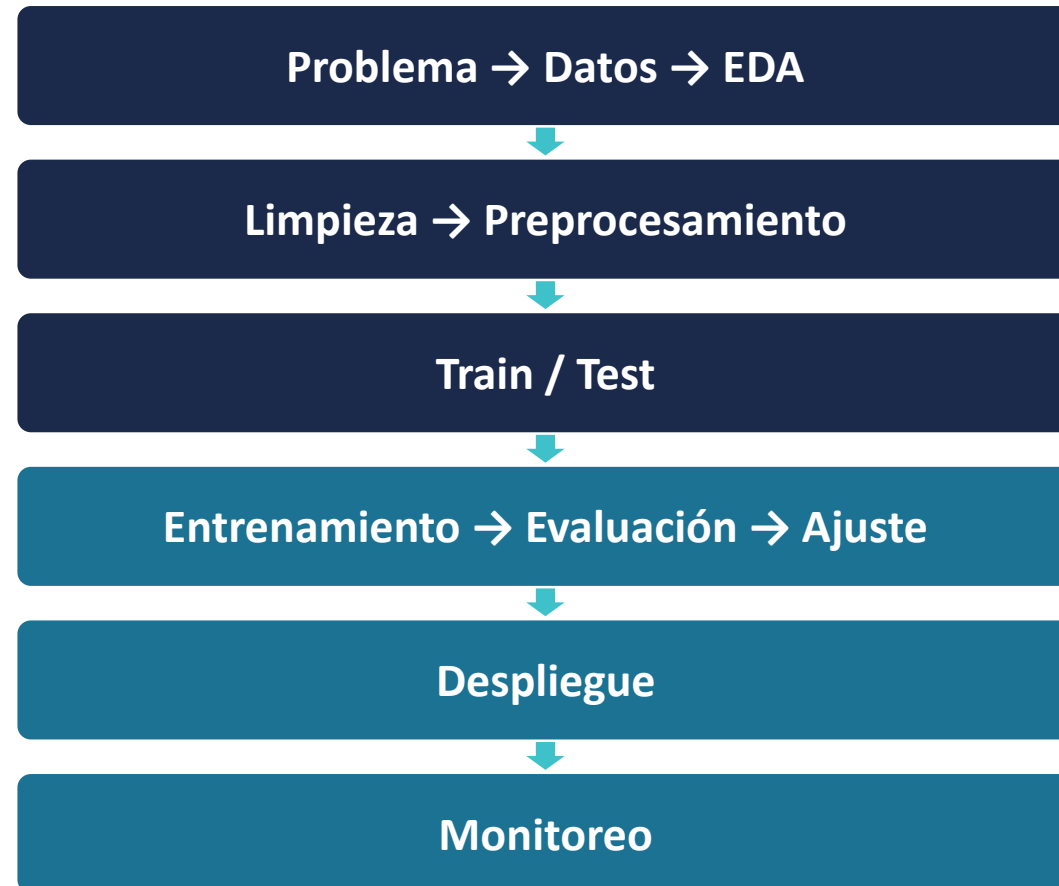
Modelo ideal - Buena generalización

Un **modelo ideal** logra aprender los patrones relevantes de los datos sin memorizar los ejemplos de entrenamiento ni ser demasiado simple.

Comportamiento

- Lo importante es que tenga un rendimiento similar y adecuado tanto en los datos conocidos como en datos nuevos.

De la definición del problema al monitoreo en producción



Machine Learning en la práctica



Finanzas

Detección de fraude, scoring crediticio



Salud

Apoyo diagnóstico, predicción de riesgo



Comercio

Recomendadores, pricing dinámico



Industria

Mantenimiento predictivo, control de calidad



Ciberseguridad

Detección de anomalías e intrusiones



Recomendadores

Sugerencia de productos y contenido



Visión artificial

Reconocimiento de imágenes y objetos



NLP

Procesamiento de lenguaje natural — tema del sábado

Biblioteca para cada etapa del pipeline



No compiten entre sí: se combinan dentro de un mismo pipeline, desde la limpieza de datos hasta el entrenamiento de una red neuronal.

SCIPY: cómputo científico sobre NumPy

*Biblioteca de Python construida sobre NumPy que añade **funciones especializadas para estadística, optimización, álgebra lineal y otras tareas científicas.***

- **scipy.stats:** pruebas estadísticas (p. ej. Shapiro-Wilk, ya usado en sesiones anteriores), distribuciones de probabilidad.
- **scipy.optimize:** algoritmos de optimización y búsqueda de mínimos/máximos.
- **scipy.linalg:** álgebra lineal avanzada (más completa que la de NumPy).
- **scipy.spatial:** cálculo de distancias y estructuras espaciales.

En el pipeline de ML, SciPy suele usarse en la etapa de análisis exploratorio y validación estadística, antes de entrenar un modelo.

Scikit-Learn: el estándar del ML tradicional

Biblioteca de Python que ofrece una interfaz consistente para **preprocesar datos, entrenar y evaluar modelos de Machine Learning tradicional**.

- **preprocessing:** escalado, codificación de variables categóricas, imputación de valores faltantes.
- **model selection:** división train/test, validación cruzada, búsqueda de hiperparámetros.
- **Algoritmos:** clasificación, regresión y clustering bajo una misma interfaz (fit / predict).
- **métricas:** evaluación de desempeño (accuracy, precision, recall, error cuadrático medio, etc.).
- **pipelines:** encadenan preprocesamiento y modelo en un solo flujo reproducible.

PyTorch: motor de Deep Learning

Biblioteca de Python para construir y entrenar redes neuronales profundas, con soporte nativo para cómputo en GPU.

- **Tensor:** la estructura de datos base — similar a un arreglo de NumPy, pero optimizada para GPU.
- **GPU:** permite acelerar drásticamente el entrenamiento de modelos grandes.
- **Autograd:** calcula automáticamente los gradientes necesarios para el ajuste de parámetros.
- **Loss y Optimizer:** la función de pérdida guía el aprendizaje; el optimizador ajusta los parámetros para reducirla.
- **Backpropagation:** el mecanismo que propaga el error hacia atrás en la red para actualizar cada capa.

La **normalización** es una técnica de **preprocesamiento de datos** que transforma los valores de una variable para llevarlos a una escala comparable, evitando que una característica domine a otras debido únicamente a su magnitud.

Por ejemplo, si tenemos:

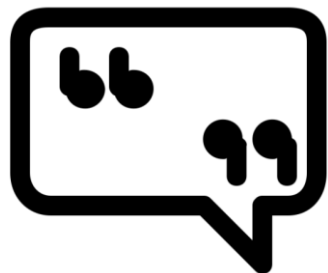
Edad $\rightarrow 18 - 80$

Ingresos $\rightarrow 1,000 - 20,000$

Los ingresos tienen valores mucho mayores. Una transformación puede llevarlos a una escala común, por ejemplo:

Edad $\rightarrow 0 - 1$

Ingresos $\rightarrow 0 - 1$



“Los sistemas de información no solo mueven datos... mueven ideas, decisiones y el futuro de las organizaciones”